

Lab 1 – Setting Up Your Professional Python Development Environment

Lab Outcomes:

- Understand what software construction means, and why professional tooling and workflow discipline reduce the cost of poorly constructed software.
- Install Python 3.12 and confirm that it runs correctly on your machine.
- Install Visual Studio Code and configure it for Python development.
- Create and use a Python virtual environment to keep every project's dependencies isolated.
- Install and verify the course's quality toolchain: ruff, mypy and pytest.
- Install Git, configure your author identity, and create a personal GitHub account and repository.
- Set up the standard project folder layout that you will reuse and extend throughout the semester.

Introduction

Welcome to your first lab of Software Construction & Development! Before we write a single line of real code, we need to set up a proper development environment on your machine — the same kind of environment a professional software engineer uses every day. Getting this right now will save you a great deal of frustration later, because every remaining lab, and the semester project, is built directly on top of what you install today.

Please work through each step below carefully and in order. If something does not work exactly as described, that is completely normal - operating systems differ slightly from one machine to another - so just ask me for help rather than skipping the step. All of the tools used in this course are free and open source; you will not need to buy any licence or set up any paid cloud account.

You must bring your own laptop to every lab session, and all of your laboratory work will be committed to your personal Git repository, so please complete every step in this lab yourself, on your own machine.

Tools and Software Required

The table below lists everything you need for this lab. The first four items are installed directly from their websites; the last three (ruff, mypy and pytest) are Python packages that you will install later in this lab, once your virtual environment is ready.

Tool	What it is used for	Official source
Python 3.12	The programming language used in every lab and the semester project.	python.org/downloads
Visual Studio Code	The code editor you will use to write, run and debug your Python code.	code.visualstudio.com
Git	Version control tool used to track and submit every change you make.	git-scm.com/downloads
GitHub account	Hosts your personal repository, which your labs and viva are graded from.	github.com/join
ruff	Lints and formats your Python code automatically.	installed via pip, Step below
mypy	Checks your code for type errors before you ever run it.	installed via pip, Step below
pytest	Runs the automated tests you write for your own code.	installed via pip, Step below

Python 3.12

Python is the programming language you will use for every lab and for the semester project. This course specifically uses Python 3.12, so please install this version even if an older or newer version is already on your machine.

Step 1: Go to python.org/downloads and download the Python 3.12 installer for your operating system (Windows, macOS or Linux).

Step 2: Windows only: on the first installer screen, tick the checkbox labelled “Add python.exe to PATH” before clicking Install Now. This checkbox is easy to miss but very important — without it, Python will not run from a terminal.

Step 3: macOS: run the downloaded .pkg file and follow the on-screen instructions, accepting the defaults.

Step 4: Linux: most distributions already include Python. First check the version already installed; only install a newer one through your package manager if it is older than 3.12.

Step 5: Open a terminal (Command Prompt or PowerShell on Windows, Terminal on macOS/Linux) and confirm the installation by running the command below. You should see Python 3.12.x printed back.

```
python --version
```

On macOS and Linux, you may need to use python3 instead of python:

```
python3 --version
```

Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight code editor with excellent Python support. You are welcome to use another editor if you are already comfortable with one, but all in-class demonstrations will use VS Code, so we strongly recommend it, especially if this is new territory for you.

Step 1: Download the installer from code.visualstudio.com for your operating system and run it with the default options.

Step 2: Open VS Code, then open the Extensions view (Ctrl+Shift+X on Windows/Linux, Cmd+Shift+X on macOS).

Step 3: Search for and install the official “Python” extension published by Microsoft. This also installs Pylance, which gives you live type-checking hints as you type.

Step 4: Open any project folder in VS Code and check the bottom-right corner of the window — it should show the Python interpreter VS Code is currently using. We will point this at your virtual environment shortly.

Git and GitHub

In this course, all laboratory work is committed to your own individual Git repository, and your commit history is itself part of what gets examined at your semester viva. It is therefore essential that Git is installed and configured correctly from day one.

Step 1: Download Git from git-scm.com/downloads and install it, keeping the default options.

Step 2: Open a terminal and set your author identity — this is the name and email that will be attached to every commit you make. Use your real name so your instructor can identify your work.

```
git config --global user.name "Your Full Name"

git config --global user.email "your_email@example.com"
```

Step 3: If you do not already have one, create a free account at github.com/join. Use an email address that you check regularly.

Step 4: On GitHub, create a new repository named `software-construction-lab`. Do not initialise it with a README just yet — you will connect it from your own machine later in this lab.

Step 5: Confirm Git is installed correctly by running the command below.

```
git --version
```

Creating a Virtual Environment and Project Layout

A virtual environment is an isolated, self-contained copy of Python used for a single project. It keeps the packages one project needs separate from every other project on your machine, which avoids version conflicts and makes your work reproducible on someone else's machine — including your instructor's, when your lab is being marked.

Step 1: Create a folder for this course and move into it.

```
mkdir software-construction-lab

cd software-construction-lab
```

Step 2: Create a virtual environment named `.venv` inside this folder.

```
python -m venv .venv
```

Step 3: Activate the virtual environment. The exact command depends on your operating system.

Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

macOS / Linux:

```
source .venv/bin/activate
```

Once activated, you should see `(.venv)` appear at the start of your terminal prompt. Keep the virtual environment active for the rest of this lab, and every time you work on this project from now on. You can turn it off at any time with the `deactivate` command.

Step 4: Inside the project folder, create the standard layout you will reuse for the rest of the semester:

- `src/` — folder for your actual source code
- `tests/` — folder for your automated tests
- `README.md` — a short description of the project

The Quality Toolchain — ruff, mypy and pytest

These three tools form the backbone of professional Python development, and you will use all three in almost every remaining lab this semester:

- `ruff` checks your code style and formatting and points out common mistakes.
- `mypy` checks that your code respects the type hints you write, catching a whole class of bugs before you ever run the program.
- `pytest` runs the automated tests you write to prove your code behaves correctly.

Step 1: With your virtual environment still active, install all three tools with a single command.

```
pip install ruff mypy pytest
```

Step 2: Verify that each tool installed correctly by checking its version.

```
ruff --version
```

```
mypy --version
```

```
pytest --version
```

Step 3: Record your exact set of installed packages in a requirements file, so your environment can be reproduced by anyone — including yourself, on a different machine.

```
pip freeze > requirements.txt
```

At the end, run this at the root so that PYTHONPATH is set to root

```
$env:PYTHONPATH = "."
```

Class Activity

Complete the following tasks during the lab session. Keep evidence (terminal output, screenshots, and your GitHub repository link) — your instructor may ask to see any of it.

Task 1: Install Python 3.12, Visual Studio Code and Git as described above, and confirm all three with their respective --version commands.

Task 2: Create a GitHub account if you do not already have one, and create a new repository named exactly software-construction-lab.

Task 3: On your machine, create the standard project layout (src/, tests/, README.md) inside a folder named software-construction-lab, together with a virtual environment named .venv.

Task 4: Activate the virtual environment, install ruff, mypy and pytest inside it, and generate a requirements.txt that lists all three.

Task 5: Inside src/, create a file named hello.py containing one function, greet(name: str) -> str, that returns the text “Hello, {name}!”.

Task 6: Inside tests/, create a file named test_hello.py with one test function that checks greet("World") returns “Hello, World!”. Run pytest and confirm the test passes.

Task 7: Run ruff check . and mypy src/ on your project, and fix any warning either tool reports.

Task 8: Initialise a Git repository inside your project folder, make your first commit, and push it to the GitHub repository you created in Task 2.

Lab Assessment Criteria:

Lab 01 Marks distribution

	ER1	ER8	RR1	RR5
Task	2 points	4 points	2 points	2 points

Lab 01 Rubric Evaluation Guideline:

#	Qualities & Criteria	0 < Poor <= 4	4.1 < Satisfactory <= 7	7.1 < Excellent <= 10
ER1	Task/Program Completion	Minimal or no task functionality was achieved.	Some tasks were completed but a few have errors or incomplete functionality.	All tasks were completed without any errors.
#	Qualities & Criteria	0 < Poor <= 4	4.1 < Satisfactory <= 7	7.1 < Excellent <= 10
ER8	Question & Answer	Answers some questions but not confidently or based on lab task knowledge.	Answers most questions confidently and based on lab task knowledge.	Answers all questions confidently and demonstrates a deep understanding of the given lab task.
#	Qualities & Criteria	0 < Poor <= 4	4.1 < Satisfactory <= 7	7.1 < Excellent <= 10
RR1	Report Structure and Clarity	Report is poorly organised, difficult to navigate, and lacks professional formatting.	Report is mostly organised but may have some minor structural issues or formatting inconsistencies.	Report is well-organised with a clear logical flow, easy navigation, and professional formatting.
#	Qualities & Criteria	0 < Poor <= 4	4.1 < Satisfactory <= 7	7.1 < Excellent <= 10
RR5	Originality and Acknowledgement	Report contains significant plagiarism (copying content	Report mostly contains original content, but there may be instances	Report content is entirely original with proper citations and

		without attribution) or improper paraphrasing of external sources.	of paraphrasing or borrowing information without proper citation.	references for any external information used.
--	--	--	---	---